

CSE308

Operating systems

V SEMESTER - CSE

Course Instructor & Time Schedule

Course Instructor

Dr. L. Amudha, AP-III/SOC

Time Schedule – Theory – CS308

Tuesday – 6th Hour – 2.00 pm to 3.00 pm

Wednesday – 1st Hour – 8.45 am to 9.45 am

Friday – 3rd Hour – 11.00 am to 12.00 pm

Laboratory – CSE309R01

Friday – 7th and 8th Hour – 3.15 to 5.15 pm

[Venue: SOCLab4]

Course Objective

This course will help the learner to understand the modules of an operating system, their functions, process management techniques, scheduling algorithms, problems of concurrent processes and possible solutions, memory management strategies, file and IO system concepts.

Syllabus

L	T	P	C
3	0	0	3

UNIT - I

11 Periods

Introduction: What Operating systems do - Structure - Operations - Process management.
Operating system Structure: Services - User Interfaces - Systems calls and types - Systems programs - Design and implementation - Operating System Structure - System boot

UNIT - II

11 Periods

Process management: Process concept - Scheduling - Operations on processes - Inter-process communication. **Threads:** Overview - Multi-core programming - Multithreading models.
Process scheduling: Basic concepts - Scheduling criteria - Algorithms: FCFS - SJFS - Priority - RR - Multilevel queue - Multilevel feedback - Thread scheduling: Contention scope - Pthread scheduling - Multiple-processor scheduling. **Process synchronization:** Background - Critical section problem - Peterson's solution - Synchronization hardware - Mutex locks - Semaphore - Classic problems of synchronization

Syllabus

UNIT - III

11 Periods

Deadlocks: System model - Characterization - Methods for handling deadlock - Prevention - Avoidance - Detection - Recovery from deadlocks. **Memory management:** Background - Swapping - Contiguous memory allocation - Paging - Structure of page tables - Segmentation. **Virtual memory:** Background - Demand paging - Copy-on-write - Page replacement - Allocation of frames - Thrashing

UNIT - IV

12 Periods

File system: File concept - Access methods - Directory and disk structure - File system mounting - Protection - File allocation methods. **Mass Storage:** Magnetic disks - Disk structure - Disk scheduling: FCFS - SSTF - SCAN - C-SCAN - LOOK - Selection of an algorithm. **I/O systems:** Overview - I/O hardware - Application I/O interface - Kernel I/O subsystem

BOOKS

TEXTBOOKS

1. Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*, Wiley, *Ninth Edition*, 2013.

REFERENCES

1. William Stallings. *Operating Systems: Internals and Design Principles*, Pearson, *Ninth Edition*, 2017.
2. Russ Cox, Frans Kaasshoek, and Robert Morris. *X. v6: A simple, Unix-like Teaching Operating System*, MIT, 2014.
3. Achyut Godbole, and Atul Kahate. *Operating Systems*, Tata McGraw, Third Edition, 2011.

Course Learning Outcomes

- Explore the modules of an operating system and to recognize the services of operating systems
- Experiment with different CPU scheduling algorithms in order to compare and analyzes their performances
- Comprehend the causes and consequences of deadlocks and the possible options to tackle them
- Learn the concepts of memory management and different memory partitioning techniques with their merits and demerits
- Implement various and compare disk scheduling algorithms
- Appraise the file organization and I/O techniques

UNIT IV I/O Systems

(Part 3)

Unit IV – Part 3 - I/O Systems

- ✓ Overview
- ✓ I/O Hardware
- ✓ Application I/O Interface
- ✓ Kernel I/O Subsystem

Objectives

- Explore the structure of an operating system's I/O subsystem
- Discuss the principles of I/O hardware and its complexity

Overview

- Two main jobs of a computer are **I/O** and **processing**
- The role of OS in computer I/O is to manage and control I/O Operations and I/O devices
- **Flow of discussion:**
 1. Basics of I/O hardware, since the nature of hardware interface places constraints on the internal facilities of operating systems.
 2. I/O services provided by the OS
 3. How the OS bridges the gap between hardware interface and the application interface

I/O Management

- ✓ I/O management is a major component of operating system design and operation
 - ✓ The **control of I/O devices** connected to the computer is a major concern.
 - ✓ I/O devices vary greatly in their **function** and **speed**.
 - ✓ Various methods are needed to control them.
 - ✓ These methods form the I/O Subsystem of the kernel, which separates the resto of the kernel from the complexities of managing I/O devices.
- ✓ **Ports, busses, device controllers** connect various devices to a computer system
- ✓ **Device drivers** encapsulate device details
 - ✓ uniform device-access interface to I/O subsystem

Device Drivers

- Encapsulates the details and oddities of different devices, the kernel of an Operating system is structured to use device driver modules.
- The device driver presents a uniform device access interface to the I/O Subsystem, much as system calls provide a standard interface between the application and operating system

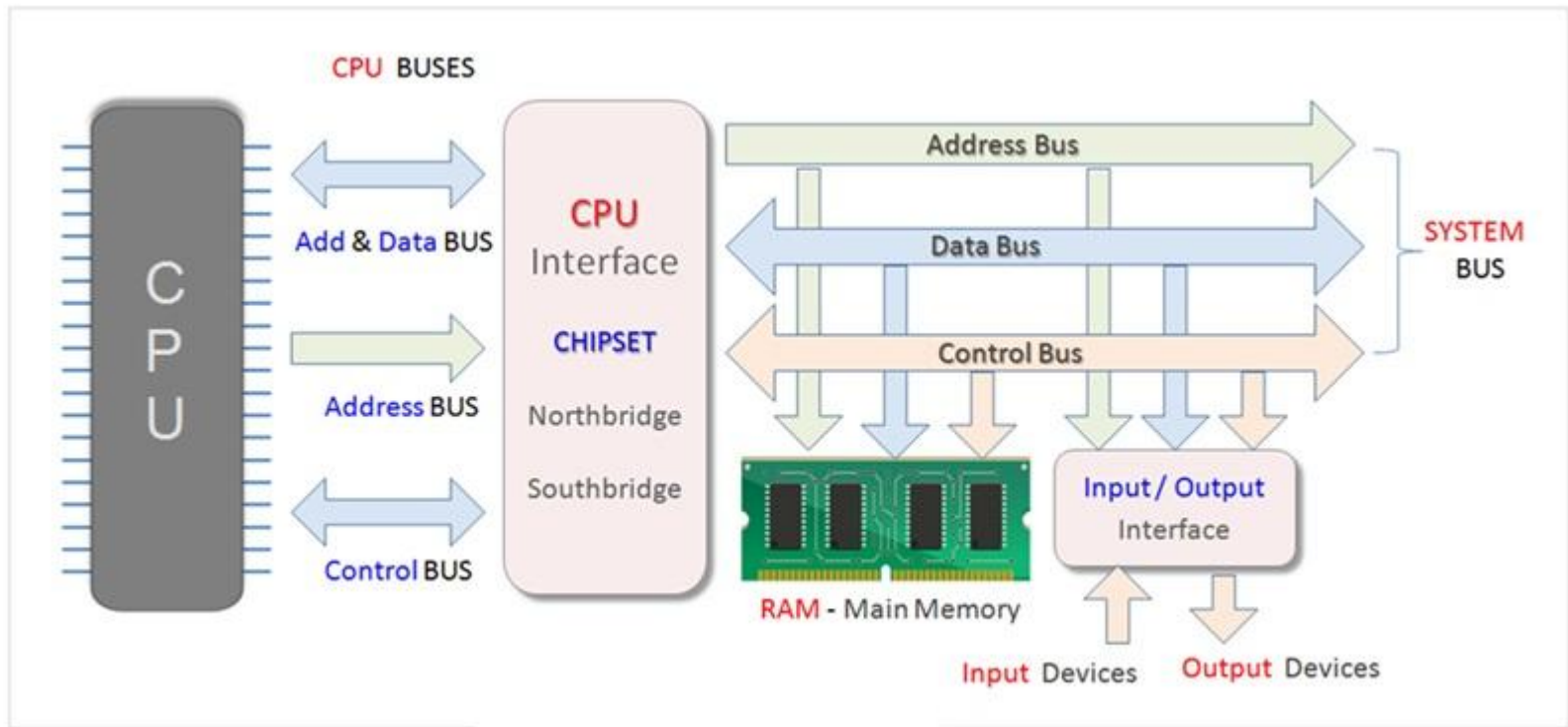
I/O Hardware

- ✓ Incredible variety of I/O devices
 - ✓ Storage (disks, tapes)
 - ✓ Transmission (n/w connection, Bluetooth)
 - ✓ Human-interface (screen, keyboard, mouse)
- ✓ Common concepts to understand
 - ✓ How the devices are attached
 - ✓ How the software can control the hardware
- ✓ A device communicates with a computer system by sending signals over a cable or wireless

Bus

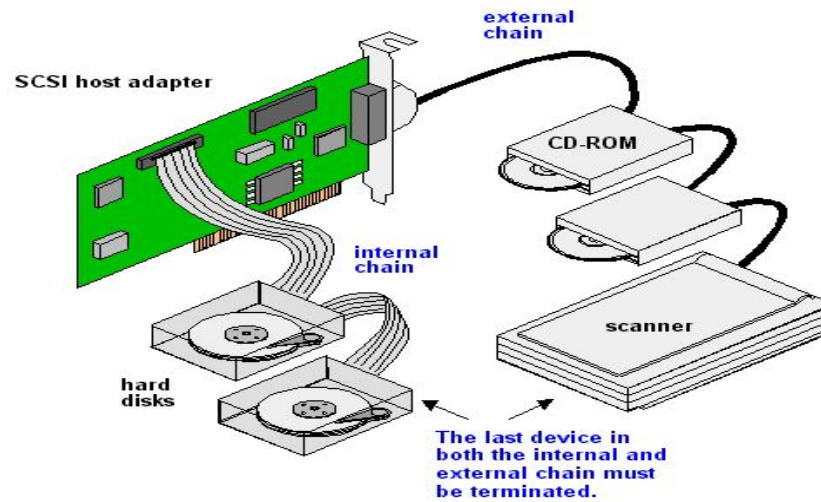
- ✓ **The device communicates with the machine via a connection point or **port** (Eg. serial port)**
- ✓ If devices share a common set of wires, the connection is called a Bus.
- ✓ Bus is a set of wires and rigidly defined protocol that specifies a set of messages that can be sent on the wires.
- ✓ In terms of electronics, the messages are conveyed by patterns of electrical voltages applied to the wires with defined timings.

BUS

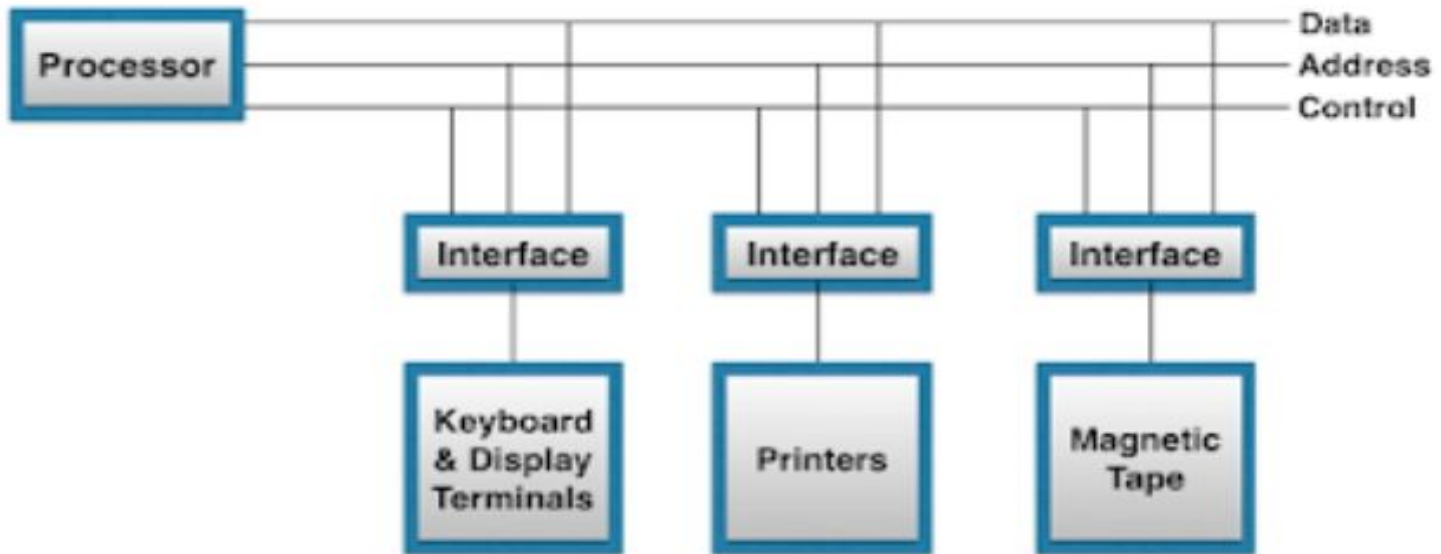


Bus – Daisy chain

- ✓ **Interconnection of set of devices together**
- ✓ **Daisy chain** or shared direct access
- ✓ When device A has a cable that plugs into device B and device B has a cable that plugs into device C and device C plugs into a port on the computer, this arrangement is called a daisy chain
- ✓ Buses are used widely in computer architecture and vary in their signaling methods, speed, throughput and connection methods



Buses



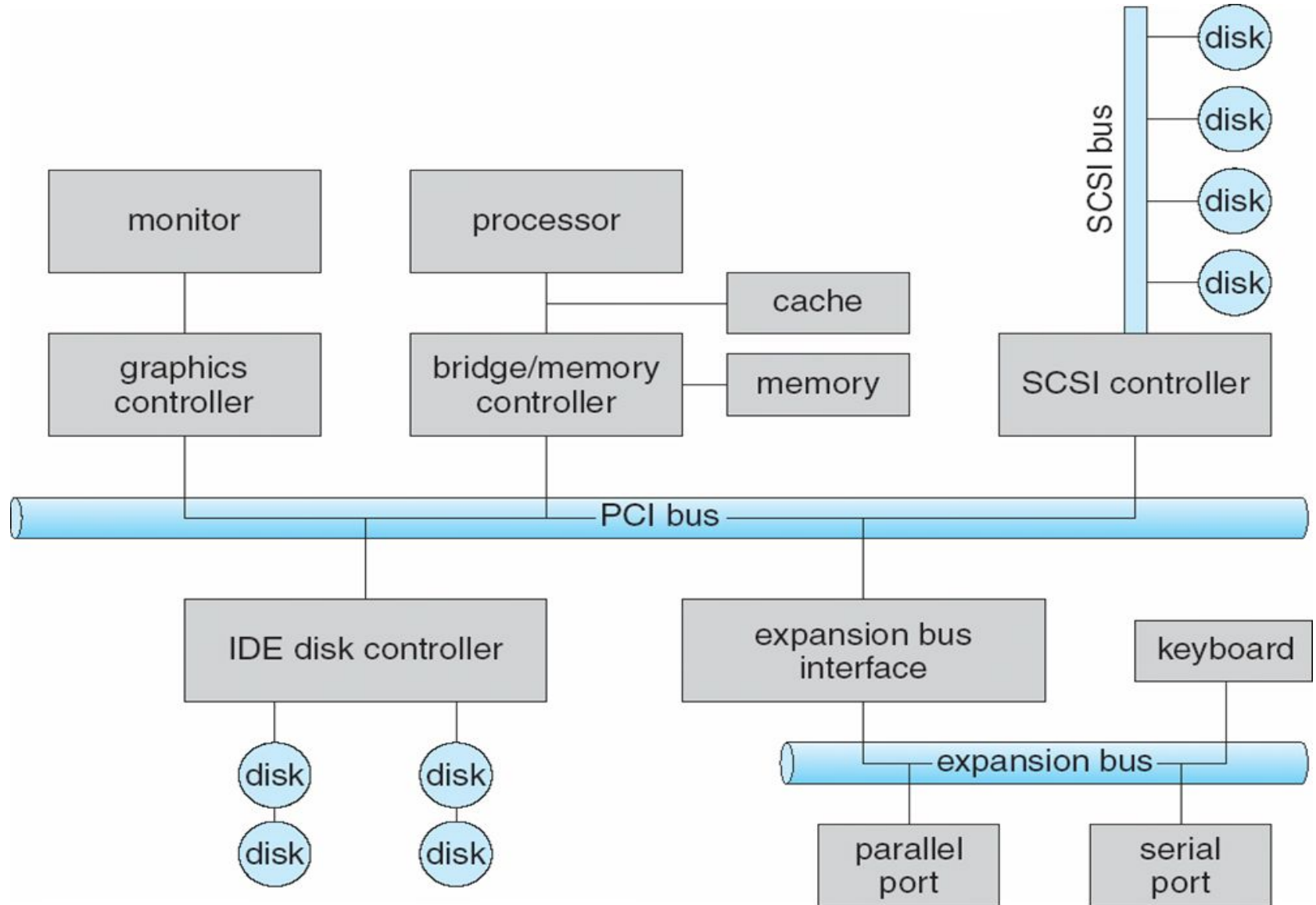
✓ Interface

- Decodes the device address (device code)
- Decodes the commands (operation)
- Provides signals for the peripheral controller
- Synchronizes the data flow and supervises the transfer rate between peripheral and CPU or Memory

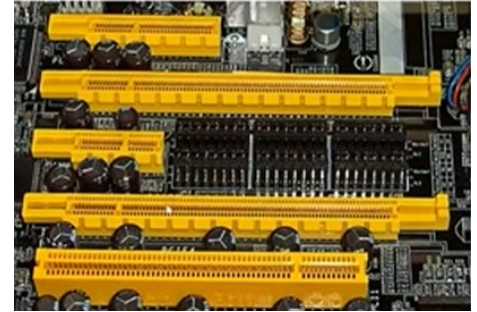
✓ 4 types of signals (an interface can receive)

✓ (control, status, data I/P, data O/P)

A Typical PC Bus Structure



PCI Bus



- ✓ **PCI** bus are common in PCs and servers
- ✓ They connect the processor-memory subsystem to fast devices
- ✓ **Expansion bus** connects relatively slow devices such as keyboard and serial and USB ports.
- ✓ SCSI (Small Computer System Interface) bus plugged into a SCSI Controller, with 4 disks in the diagram
- ✓ Other common buses include PCI Express (PCIe)
- ✓ **HyperTransport** bus has a throughput of upto 25 GB per second

Controller

- ✓ **Controller (host adapter)** – collection of electronics that can operate port, bus, device
- ✓ A serial-port controller is a simple device controller.
- ✓ It is a single chip (or portion of a chip) in the computer that controls the signals on wires of a serial port.
 - ✓ Contains processor, microcode, private memory, bus controller, etc
 - ✓ Some talk to per-device controller with bus controller, microcode, memory, etc

Hardware devices



Modem



NIC



Repeater



Hub



Switch



Router

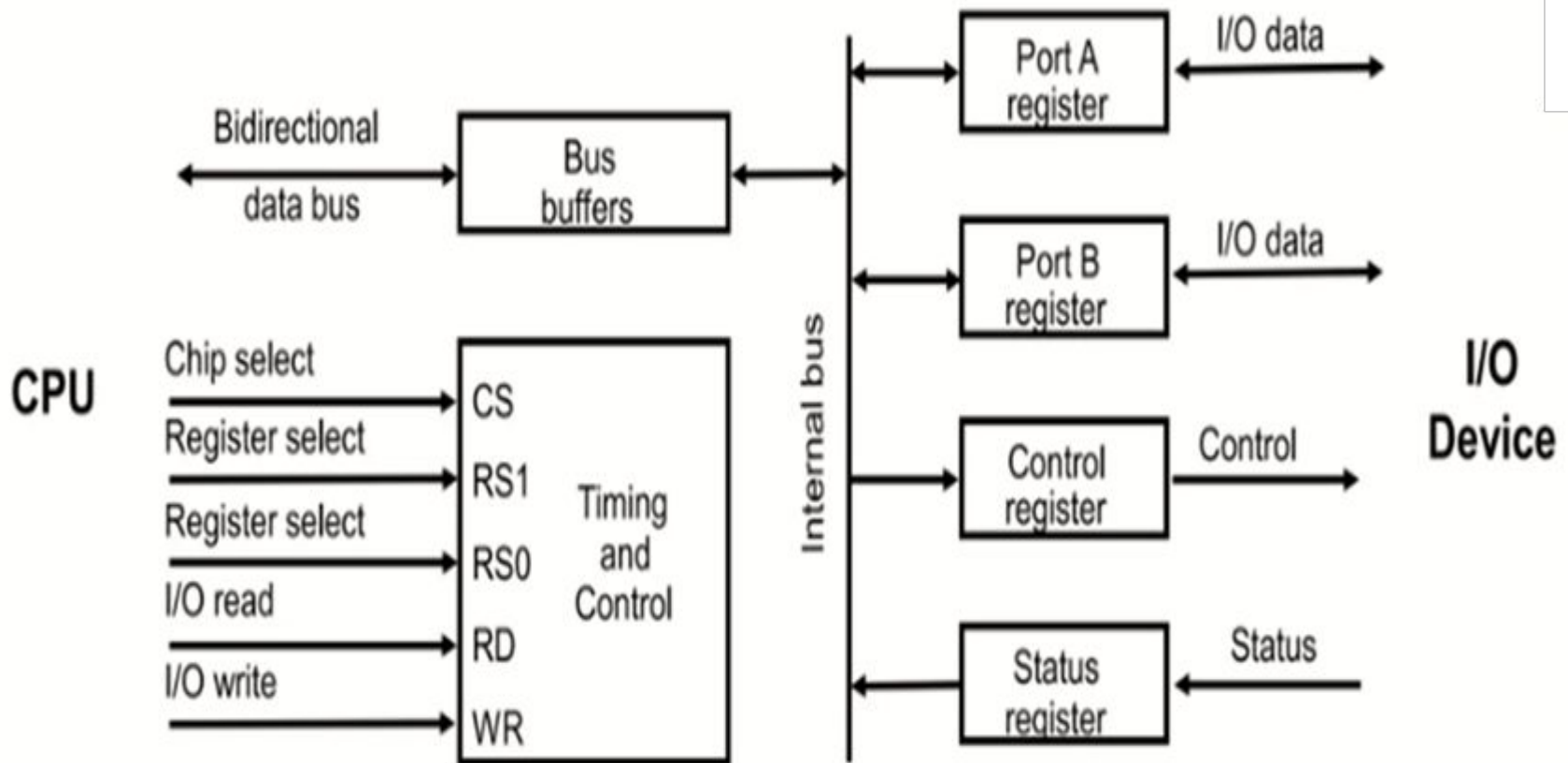


Bridge



Gateway

How a processor gives commands and data to a controller to accomplish an I/O Transfer



I/O Hardware

Registers – The controller has one or more registers for data and control signals. [Data-in register, data-out register, status register, control register]

Two ways of communication

1. Isolated I/O or Direct I/O instructions

The processor communicates with the controller by reading and writing bit patterns in these registers

Through special I/O instructions that specify the transfer of a byte or word to an I/O port address

The I/O instruction triggers bus lines to select the proper device and to move bits into or out of a device register

I/O Hardware (Cont.)

2. Memory-mapped I/O

- ✓ Device data and command registers mapped to processor address space
- ✓ Especially for large address spaces (graphics)
- ✓ CPU executes I/O requests using the standard transfer instructions to read and write the device control registers at their mapped locations in physical memory
- ✓ Some systems use both techniques.

Device I/O Port Locations on PCs (partial)

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)

Polling

- For each byte of I/O
 1. Read busy bit from status register until 0
 2. Host sets read or write bit and if write copies data into data-out register
 3. Host sets command-ready bit
 4. Controller sets busy bit, executes transfer
 5. Controller clears busy bit, error bit, command-ready bit when transfer done
- Step 1 is **busy-wait** cycle to wait for I/O from device
 - Reasonable if device is fast
 - But inefficient if device slow
 - CPU switches to other tasks?
 - 4 But if miss a cycle data overwritten / lost

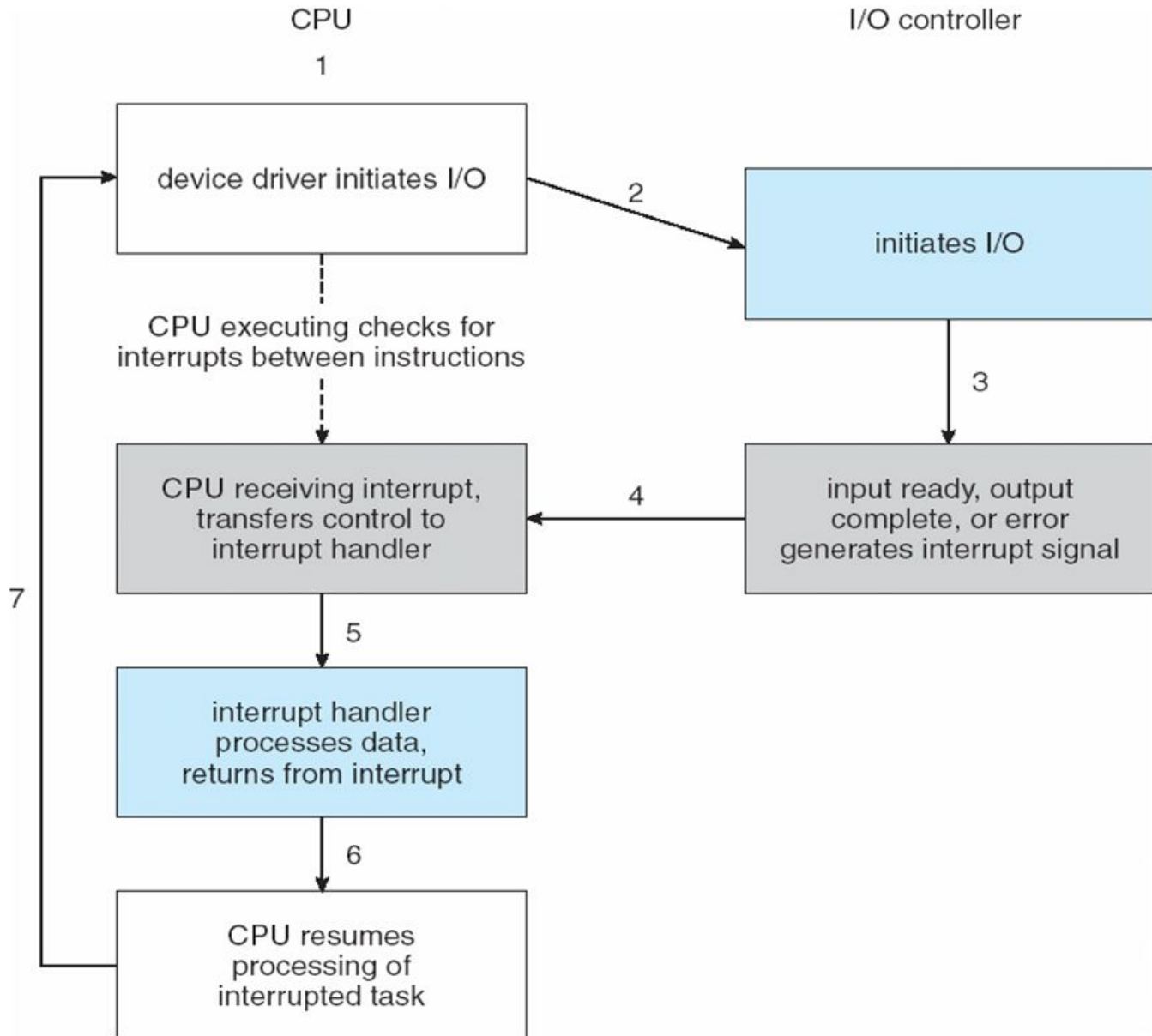
Interrupts

- ✓ **Polling can happen in 3 instruction cycles**
 - ✓ Read status, logical-and to extract status bit, branch if not zero
 - ✓ How to be more efficient ?
- ✓ **Interrupt :** A hardware mechanism that enables a device to notify the CPU
- ✓ CPU **Interrupt-request line** triggered by I/O device
 - ✓ Checked by processor after each instruction
- ✓ **Interrupt handler** receives interrupts
 - ✓ **Maskable** to ignore or delay some interrupts
- ✓ **Interrupt vector** to dispatch interrupt to correct handler
 - ✓ Context switch at start and end
 - ✓ Based on priority
 - ✓ Some **nonmaskable**
 - ✓ Interrupt chaining if more than one device at same interrupt number

Interrupt request lines

- Most CPUs have 2 interrupt request lines.
- One is the **non-maskable** interrupt, which is reserved for events such as **unrecoverable memory errors**.
- The second interrupt line is **maskable**: it can be turned off by the CPU before execution of critical instruction sequences that must not be interrupted.
- The maskable interrupt is used by device controllers to request service

Interrupt-Driven I/O Cycle



Intel Pentium Processor Event-Vector Table

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Interrupts (Cont.)

- Interrupt mechanism also used for **exceptions**
 - Terminate process, crash system due to hardware error
- Page fault executes when memory access error
- System call executes via **trap** to trigger kernel to execute request
- Multi-CPU systems can process interrupts concurrently
 - If operating system designed to handle it
- Used for time-sensitive processing, frequent, must be fast

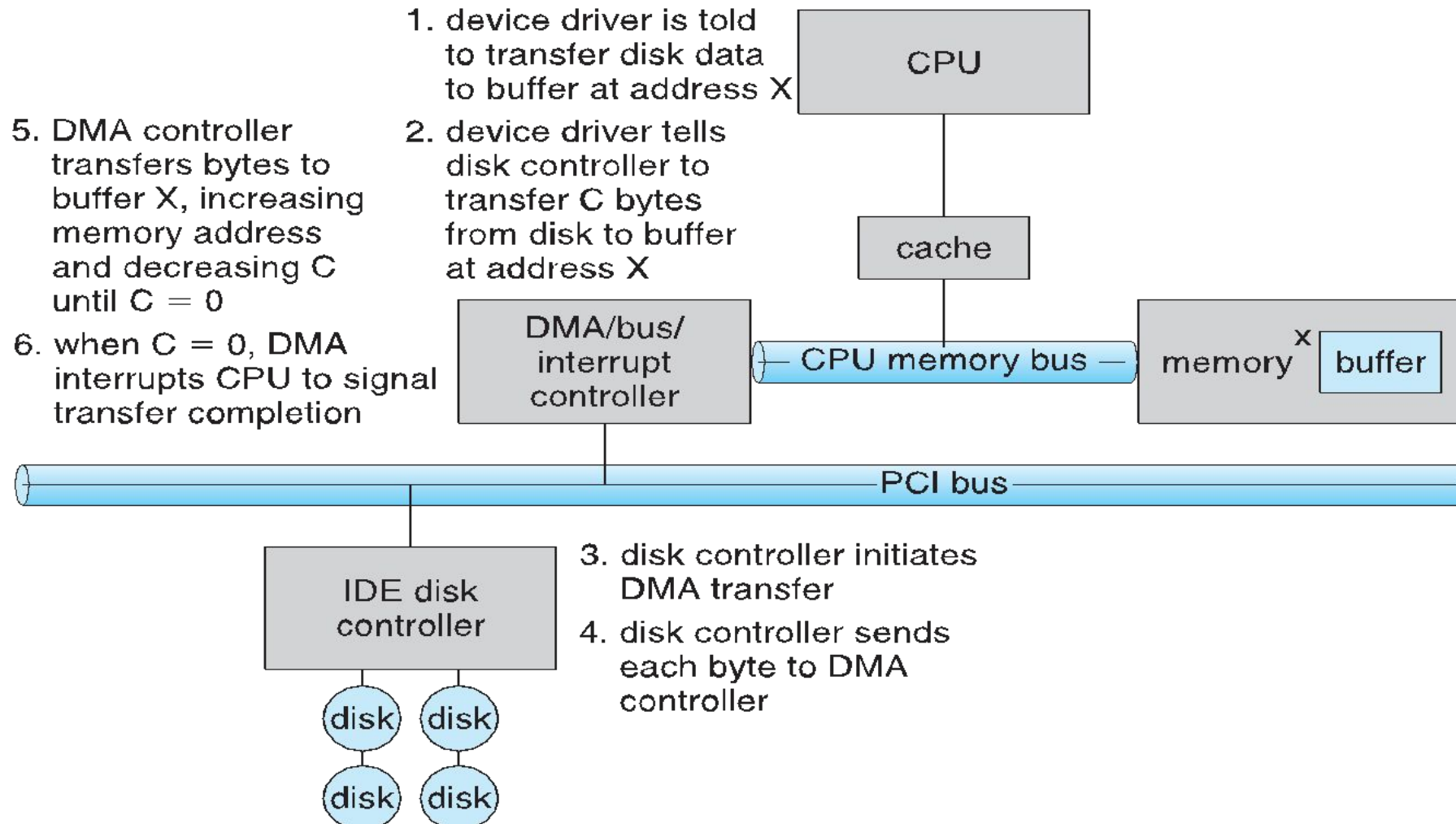
Trap

- Implementation of **System Calls**
- Usually a program uses **library calls** to issue system calls
- The library routines **check the arguments** given by the application, build a data structure to convey the arguments to the kernel, and then execute a special instruction called **Software interrupt** or **Trap**.
- This instruction has an **operand that identifies** the desired **kernel service**
- When a process executes the trap instruction, the interrupt hardware **saves the state of the user code, switches to kernel mode** and **dispatches to the kernel routine** that implements the requested service.
- The trap is given a **relatively low interrupt priority** compared with those assigned to device interrupts.

Direct Memory Access

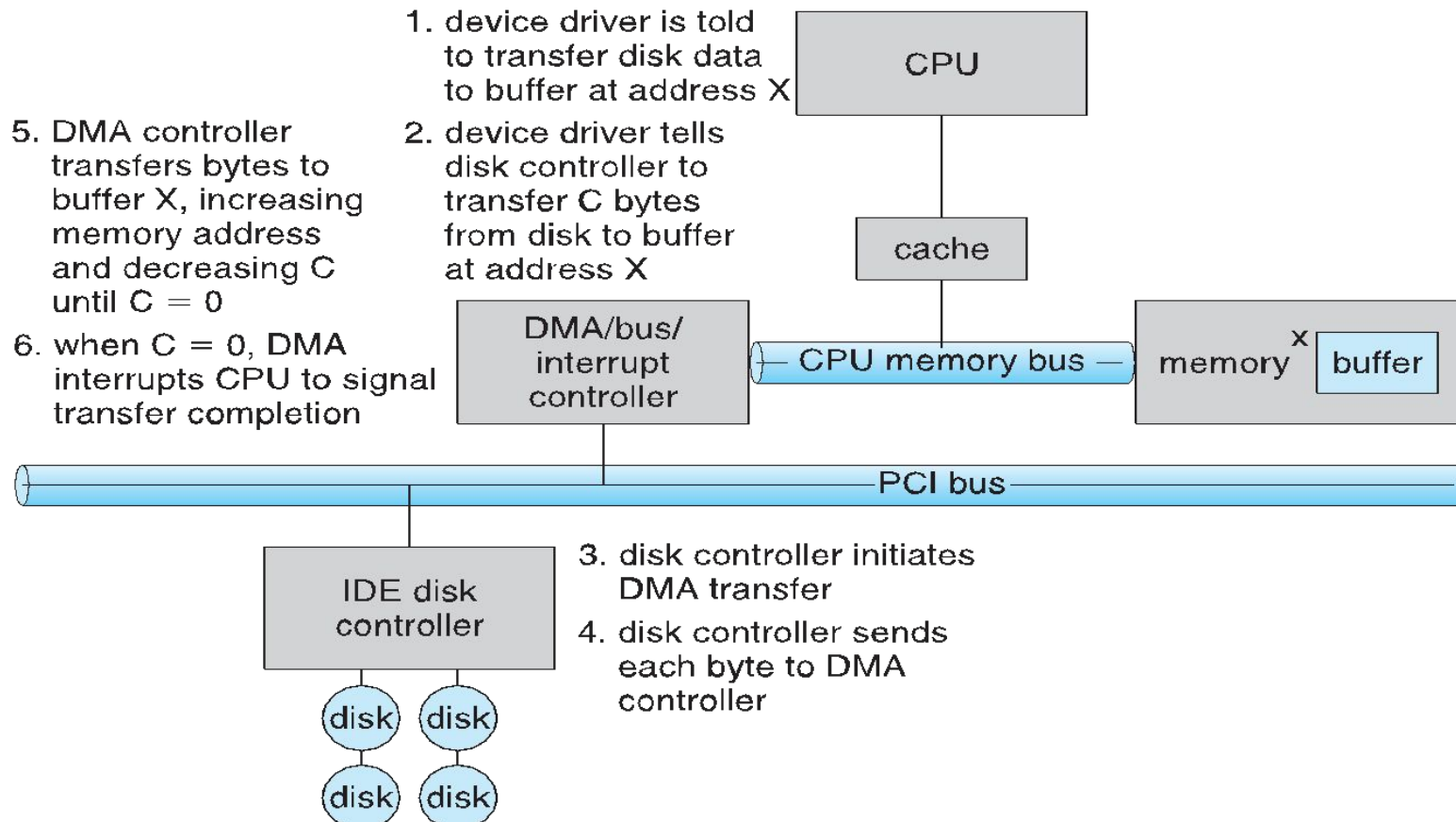
- Used to avoid **programmed I/O** (one byte at a time) for large data movement
- Requires **DMA** controller
- Bypasses CPU to transfer data directly between I/O device and memory
- OS writes DMA command block into memory
 - Source and destination addresses
 - Read or write mode
 - Count of bytes
 - Writes location of command block to DMA controller
 - Bus mastering of DMA controller – grabs bus from CPU
 - 4 **Cycle stealing** from CPU but still much more efficient
 - When done, interrupts to signal completion

6 Step Process to Perform DMA Transfer



Cycle Stealing

When the DMA controller seizes the memory bus, CPU is momentarily prevented from accessing main memory - **Cycle stealing**



Application I/O Interface

- I/O system calls encapsulate device behaviors in generic classes.
- Device-driver layer hides differences among I/O controllers from kernel.
- A form of abstraction, the detailed differences can be overlooked and grouped to few generic kinds.
- Each OS has its own I/O subsystem structures and device driver frameworks - **Interface**
- **Devices vary in many dimensions**
 - **Character-stream** or **block**
 - **Sequential** or **random-access**
 - **Synchronous** or **asynchronous** (or both)
 - **Sharable** or **dedicated**
 - **Speed of operation**
 - **read-write, read only, or write only**

Characteristics of I/O Devices (Cont.)

- Subtleties of devices handled by device drivers
- Broadly I/O devices can be grouped by the OS into
 - ✓ Block I/O
 - ✓ Character I/O (Stream)
 - ✓ Memory-mapped file access
 - ✓ Network sockets
- For direct manipulation of I/O device specific characteristics, usually an escape / back door
 - ✓ Unix `ioctl()` call to send arbitrary bits to a device control register and data to device data register

Block and Character Devices

- The block-device interface captures all the aspects necessary for accessing disk drives and other block-oriented devices.
- The device is expected to understand commands such as `read()` and `write()`. If it is a random-access device. It is also expected to have a `seek()` command to specify which block to transfer next.
- Applications normally access such a device through a file system interface.
- The operating user may prefer to access a block device as a simple linear array of blocks.
- This mode of access is called raw I/O.

Block and Character Devices

- **Block devices** include disk drives
 - Commands include read, write, seek
 - **Raw I/O**, **direct I/O**, or file-system access
 - Memory-mapped file access possible
 - 4 File mapped to virtual memory and clusters brought via demand paging
 - DMA
- **Character devices** include keyboards, mouse, serial ports
 - Commands include `get()`, `put()`
 - Libraries layered on top allow line editing

Characteristics of I/O Devices

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read–write	CD-ROM graphics controller disk

Blocking and Non-blocking I/O transfers

- System call makes a choice – **blocking** and **non blocking**.

Blocking I/O

- When an application issues a blocking system call, the execution of the application is suspended.
- The application is moved from the operating system's run queue to a wait queue.
- After the system call completes, the application is moved back to the run queue.
- When it resumes execution, it will receive the values returned by the system call

Non-blocking I/O

- Some user-level processes need **nonblocking I/O**.
- One example is a **user interface that receives keyboard and mouse input while processing and displaying data on the screen**.
- Another example is a **video application** that reads frames from a file on disk while simultaneously decompressing and displaying the output on the display.
- One way an application writer can **overlap execution** with I/O is to write a **multi threaded application**. Some threads can perform blocking systems calls, while others continue executing.

Kernel I/O Subsystem

- Kernels provide **many services** related to I/O
- **Scheduling, Buffering, Caching, Spooling, Device reservation, Error handling** are all services provided by kernel I/O subsystem.
- It is also responsible for **protecting itself** from errant processes and malicious users.

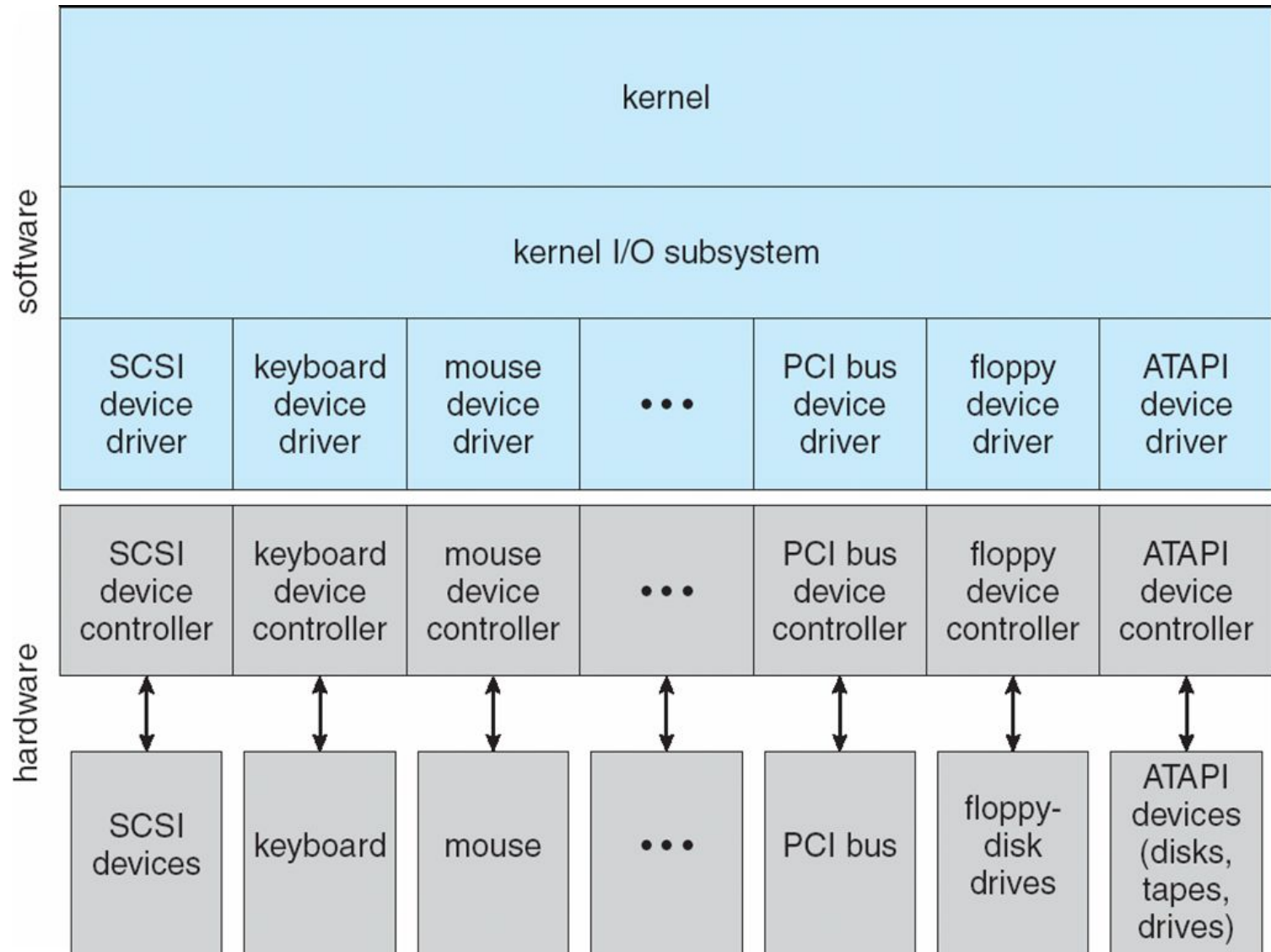
Kernel I/O Subsystem

- **Caching** - faster device holding copy of data
 - Always just a copy
 - Key to performance
 - Sometimes combined with buffering
- **Spooling** - hold output for a device
 - If device can serve only one request at a time
 - i.e., Printing
- **Device reservation** - provides exclusive access to a device
 - System calls for allocation and de-allocation
 - Watch out for deadlock

Kernel I/O Subsystem

- **Scheduling**
 - Some I/O request ordering via per-device queue
 - Some OSs try fairness
 - Some implement Quality Of Service (i.e. IPQOS)
- **Buffering** - store data in memory while transferring between devices
 - To cope with device speed mismatch
 - To cope with device transfer size mismatch
 - To maintain “copy semantics”
 - **Double buffering** – two copies of the data
 - 4 Kernel and user
 - 4 Varying sizes
 - 4 Full / being processed and not-full / being used
 - 4 Copy-on-write can be used for efficiency in some cases

A Kernel I/O Structure



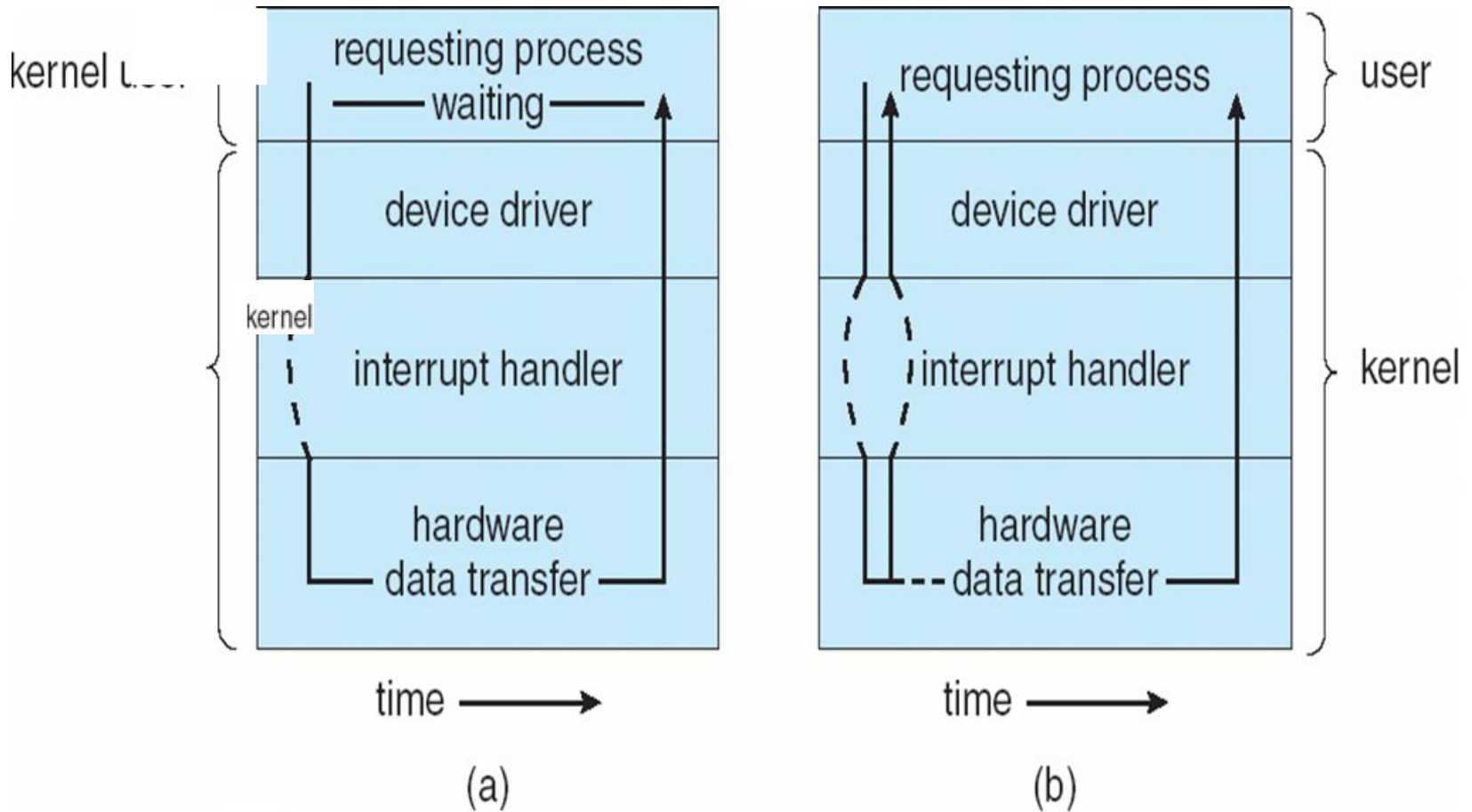
Network Devices

- Varying enough from block and character to have own interface
- Linux, Unix, Windows and many others include **socket** interface
 - Separates network protocol from network operation
 - Includes **select()** functionality
- Approaches vary widely (pipes, FIFOs, streams, queues, mailboxes)

Clocks and Timers

- Provide current time, elapsed time, timer
- Normal resolution about 1/60 second
- Some systems provide higher-resolution timers
- **Programmable interval timer** used for timings, periodic interrupts
- `ioctl()` (on UNIX) covers odd aspects of I/O such as clocks and timers

Two I/O Methods



Synchronous

Asynchronous

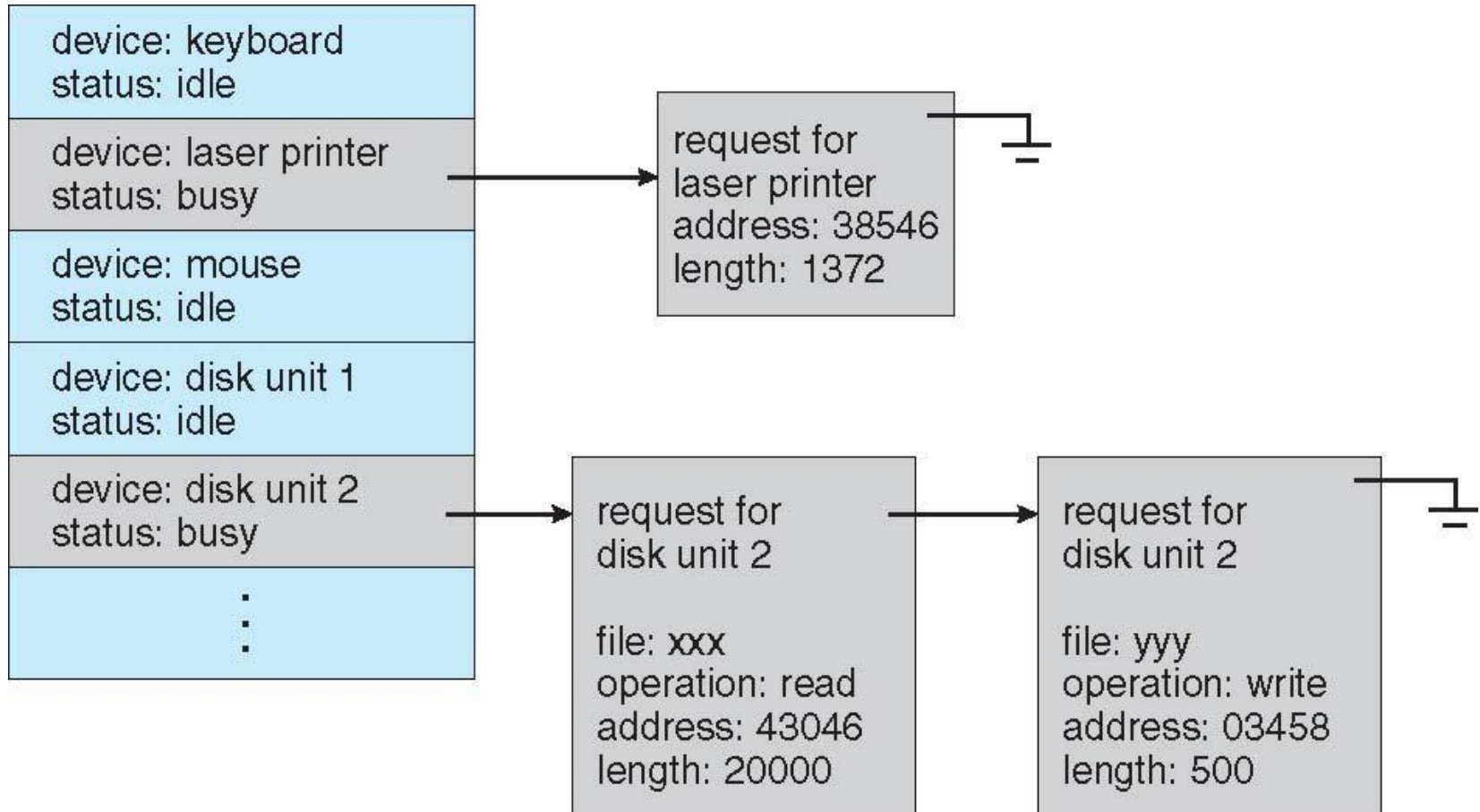
Vectored I/O

- **Vectored I/O** allows one system call to perform multiple I/O operations
- For example, Unix `readve()` accepts a vector of multiple buffers to read into or write from
- This scatter-gather method better than multiple individual I/O calls
 - Decreases context switching and system call overhead
 - Some versions provide atomicity
 - 4 Avoid for example worry about multiple threads changing data as reads / writes occurring

I/O Scheduling

- Order of servicing IO requests
- Determine a good order
- Share device access fairly
- Reduce average waiting time
- Disk scheduling is an example

Device-status Table



Buffering

A **buffer** is a memory area that stores data being transferred between 2 devices or between a device and an application.

Buffering done for 3 reasons

1. To cope with the **speed mismatch** between producer and consumer of data stream
2. To provide adaptations for devices that have **different data transfer sizes**.
3. Third use of buffering is to support **copy semantics**

Copy Semantics

Example: Suppose that an **application** has a **buffer of data** that it wishes to **write to disk**.

- It calls the **write() system call**, providing a pointer to the buffer and an **integer** specifying the number of bytes to write.
- After the system call returns, what happens if the **application changes** the contents of the buffer?
- With copy semantics, the version of the **data written to disk is guaranteed** to be the version at the time of the application call, independent of any subsequent changes in the application's buffer.
- A simple way in which the operating system can guarantee copy semantics is for the write() system call to copy the application data into a kernel buffer before returning control to the application.
- The disk write is performed from the kernel buffer, so that subsequent changes to the application buffer have no effect

Caching and Buffering

Caching and buffering are distinct functions, but sometimes a region of memory can be used for both purposes.

For instance, to preserve copy semantics and to enable efficient scheduling of disk I/O, the operating system uses buffers in main memory to hold disk data.

These buffers are also used as cache, to improve the I/O efficiency for files that are shared by applications that are written and reread frequently.

When the kernel receives a file I/O request, the kernel first accesses the buffer cache to see whether that region of the file is already available in main memory.

If it is, a physical disk I/O can be avoided or deferred.

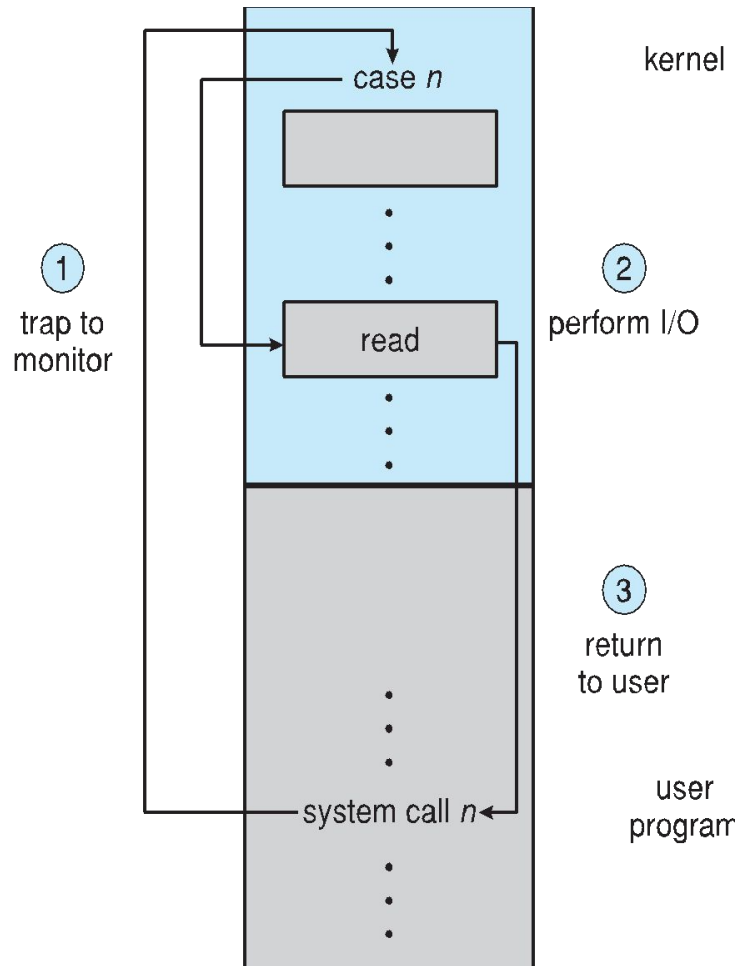
Also disk writes are accumulated in the buffer cache for several seconds, so that large transfers are gathered to allow efficient write schedules.

IO PROTECTION

Error Handling

- OS can recover from disk read, device unavailable, transient write failures
 - Retry a read or write, for example
 - Some systems more advanced – Solaris FMA, AIX
 - 4 Track error frequencies, stop using device with increasing frequency of retry-able errors
- Most return an error number or code when I/O request fails
- System error logs hold problem reports

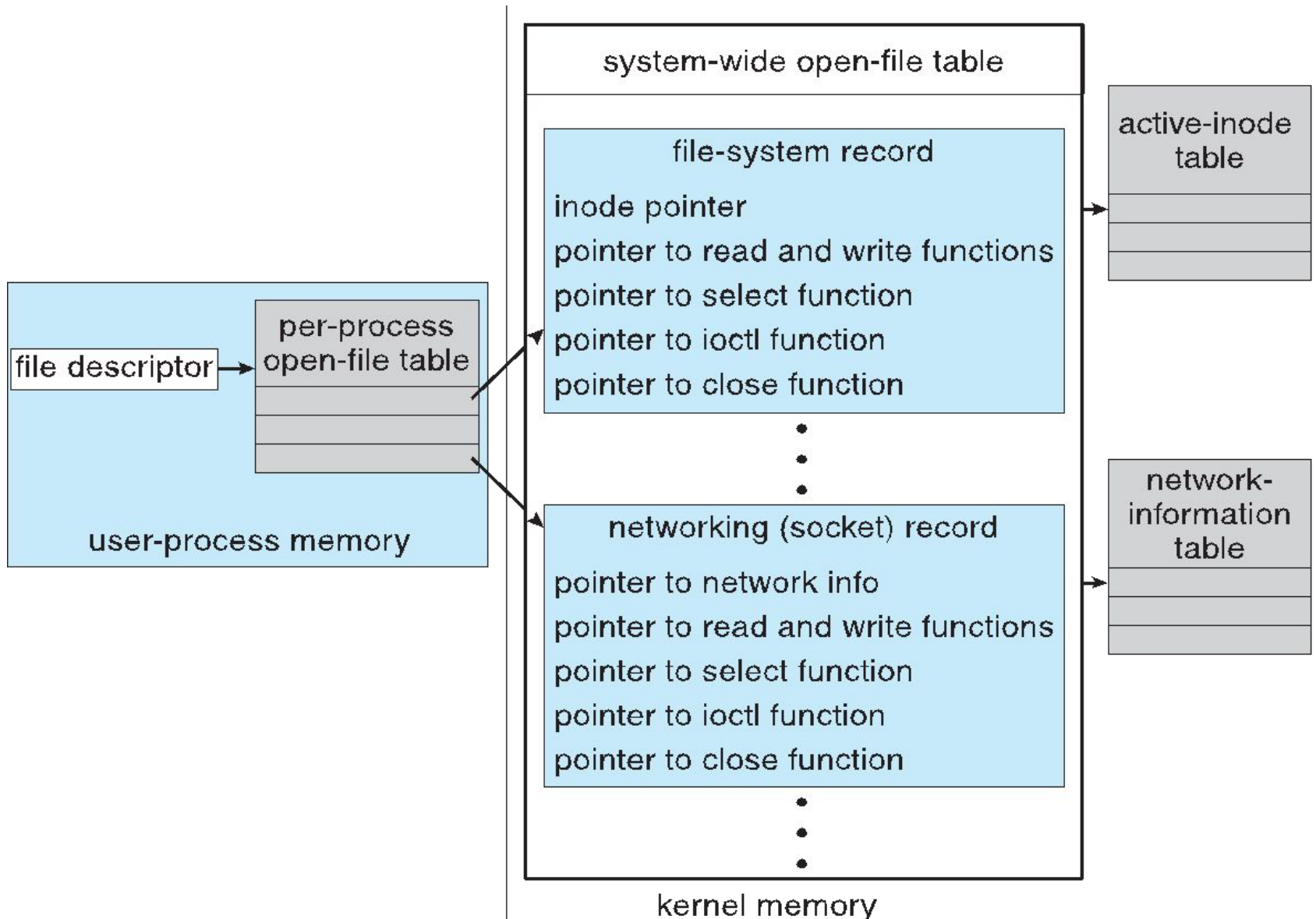
Use of a System Call to Perform I/O



Kernel Data Structures

- Kernel keeps state info for I/O components, including **open file tables, network connections, character device state**
- Many, many **complex data structures** to track buffers, memory allocation, “dirty” blocks
- Some use object-oriented methods and message passing to implement I/O
 - Windows uses message passing
 - 4 Message with I/O information passed from user mode into kernel
 - 4 Message modified as it flows through to device driver and back to process

UNIX I/O Kernel Structure



Kernel I/O Subsystem summary

- Management of the name space for files and devices
- Access control to files and devices
- Operation control (for example, a modem cannot seek())
- File-system space allocation
- Device allocation
- Buffering, caching, and spooling
- I/O scheduling
- Device-status monitoring, error handling, and failure recovery
- Device-driver configuration and initialization